FACULTY OF COMPUTING

BSc(Hons) in Software Engineering

Operating System (IT 2106)

Deshan siriwardhana (M20040117004)

Contents

1. Introduction to the Critical-Section Problem 3
2. Demonstrating the Race Condition (Unsynchronized Implementation) 3
3. Peterson's Algorithm Explained 5
4. Implementation in Java 6
5. How Peterson's Algorithm Satisfies the Three Requirements 8
   1.Mutual Exclusion 8
   2.Progress 8
   3.Bounded Waiting 8
6. Code Walkthrough 9
7.Screenshots of program output and performance comparison. 10
   1.synchronized version 10
   2. unsynchronized version 10
8. Conclusion and Reflections 11
9.References: 11

Peterson's Algorithm A Software-Based Synchronization Solution

1. Introduction to the Critical-Section Problem

The critical-section problem arises in concurrent programming when multiple threads or processes access and modify shared resources simultaneously. Without proper synchronization, race conditions occur, leading to data corruption, inconsistent states, and incorrect program behavior.

A critical section is a segment of code where shared resources are accessed. To prevent race conditions, any solution must satisfy three requirements:

I. Mutual Exclusion :Only one thread can execute its critical section at a time.
II. Progress: If no thread is in the critical section, a thread wishing to enter can proceed without unnecessary delay.
III. Bounded Waiting: No thread should wait indefinitely to enter its critical section.

2. Demonstrating the Race Condition (Unsynchronized Implementation)

Before implementing the synchronized solution, it is important to understand the problem. In this version, two threads try to update a shared counter simultaneously without any protection. This leads to a Race Condition.

Why the Race Condition Occurs:

The operation count++ looks like a single step, but in the CPU level, it consists of three distinct steps:

1. Read: The thread reads the current value of count from memory.
2. Increment: The thread adds 1 to the value.
3. Store: The thread writes the updated value back to memory.

When two threads (Thread 0 and Thread 1) run at the same time, they might both read the same initial value before either has a chance to update it. As a result, one of the updates is "lost," and the final count becomes less than the expected 200,000.

```
package UnsynchronizedVersion;
// This class holds the shared data
class Counter {
   private int count = 0;
   //  This is the Critical Section.
   //  Without synchronized, two threads will overwrite each other's work.
   public void increment() {
      count++; }
 public int getCount() {
      return count;  }
}

// This class represents a thread
class ProcessThread extends Thread {
   private Counter sharedCounter;
   public ProcessThread(Counter sharedCounter) {
  this.sharedCounter = sharedCounter;
   }

@Override
   public void run() {
// Each thread tries to add 1 to the counter 100,000 times
         for (int i = 0; i < 100000; i++) {
         sharedCounter.increment();
      }
   }
}
public class Main {
   public static void main(String[] args) {
```

```
        Counter counter = new Counter()
        // Creating two threads t1 and t2
        ProcessThread t1 = new ProcessThread(counter);
        ProcessThread t2 = new ProcessThread(counter);
        System.out.println("Starting Unsynchronized Execution...");

        t1.start();
        t2.start();
        try {
            // Waiting for threads to finish
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            System.err.println("Thread interrupted: " + e.getMessage());
        }
        // // Output will be less than 200,000 because of the race condition
        int expectedValue = 200000;
        int actualValue = counter.getCount();
        System.out.println("Expected Final Count: " + expectedValue);
        System.out.println("Actual Final Count:   " + actualValue);
        System.out.println("Difference (Lost Updates): " + (expectedValue - actualValue));
        System.out.println("Status: Race Condition Demonstrated Successfully.");
    }
}
```

3. Peterson's Algorithm Explained

Peterson's Algorithm is a software-based synchronization solution designed for two processes or threads. It uses shared variables to coordinate entry into the critical section without requiring hardware instructions or operating system support.

Key Variables:

- Boolean flag [2] : Indicates a thread's desire to enter the critical section.
- int turn : Indicates whose turn it is to enter.

Algorithm Steps,

1) Express interest by setting flag[i] = true.
2) Give turn to the other thread (turn = j).
3) Wait until either:
    - The other thread is not interested (flag[j]), or
    - It's your turn (turn == i)

4. Implementation in Java

```
package SyncronizedVersion;

class PetersonAlgorithm {
   //volatile is essential to ensure that data is exchanged instantly between two threads.
   private static volatile boolean[] flag = new boolean[2];
   private static volatile int turn = 0;

   public static void enter(int i) {
      int j = 1 - i; //ID of the other thread
      flag[i] = true; //Current thread expresses interest to enter
      turn = j;    // Give turn to the other thread

// Wait as long as the other thread needs to enter, now it's their turn.
            while (flag[j] && turn == j) {
         // Busy wait (Waiting for the critical section to become free)
      }
   }

   public static void exit(int i) {
      flag[i] = false;     // Finished, now the other thread can enter
   }
}
```

```
class Counter {
   int count = 0;

   void increment(int id) {
      PetersonAlgorithm.enter(id); // entry section
  //request to join critical section
      count++; //critical section
// critical section end
      PetersonAlgorithm.exit(id);   //exit section
   }
}
class ProcessThread extends Thread {
   Counter resource;
   int id;

   ProcessThread(Counter resource, int id) {
      this.resource = resource;
      this.id = id;
   }

   @Override
   public void run() {
      // Each thread will try to increment the counter 100,000 times.
      for (int i = 0; i < 100000; i++) {
         resource.increment(id);
      }
   }
}
public class Main {
   public static void main(String[] args) throws Exception {
      Counter resource = new Counter();
// Creating two process threads
      ProcessThread t1 = new ProcessThread(resource, 0);
      ProcessThread t2 = new ProcessThread(resource, 1);

      System.out.println("Starting Peterson's Algorithm Implementation...");

      t1.start();
      t2.start();
      // Wait for both threads to finish before  the final result
      t1.join();
      t2.join();
```

```
        // Final count should be exactly 200,000
        System.out.println("Final  Count : " + resource.count);

        if(resource.count == 200000) {
            System.out.println("Status: Success! No Race Condition.");
        } else {
            System.out.println("Status: Failed! Data inconsistency detected.");
        }
    }
}
```

5. How Peterson's Algorithm Satisfies the Three Requirements

1.Mutual Exclusion

- If both threads set flag[i] = true, the turn variable decides.
- Only one thread can satisfy the condition turn == i at a time.
- The while-loop ensures a thread waits if the other is in the critical section.

2.Progress

- If a thread exits the critical section, it sets flag[i] = false.
- The other thread will then exit the while-loop and enter.
- No thread is blocked indefinitely if the other is not competing

3.Bounded Waiting

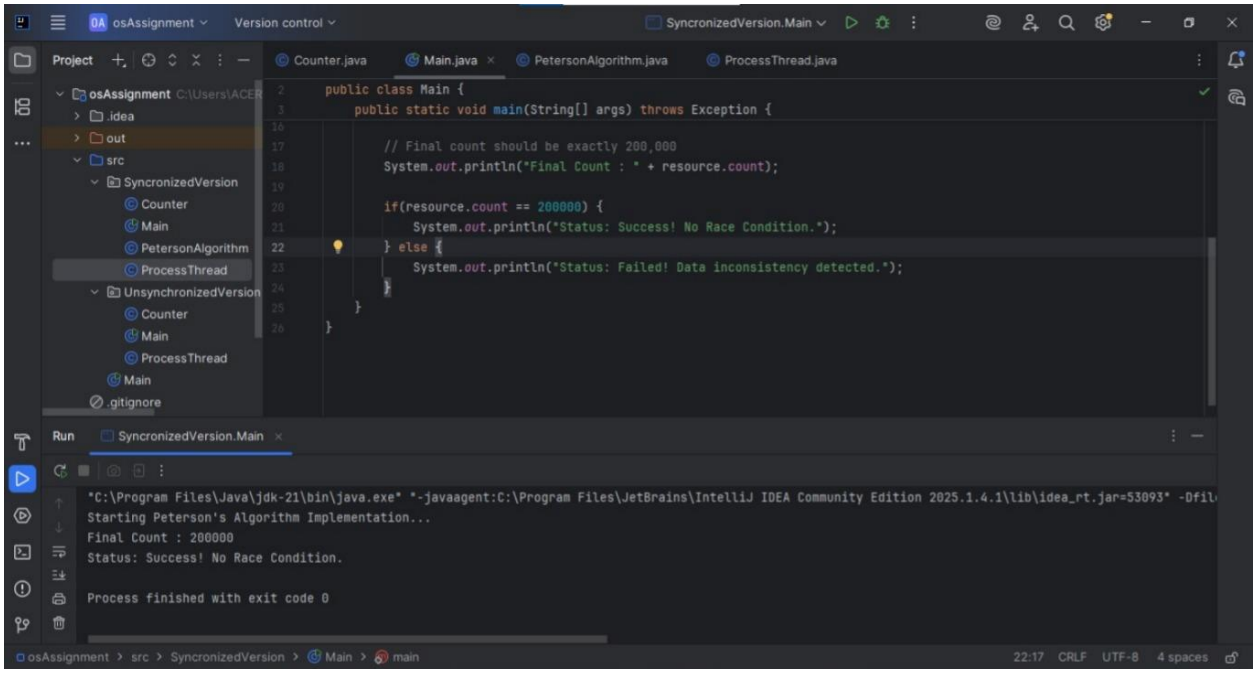
- After exiting, a thread resets flag[i] = false and passes the turn.
- The waiting thread will get turn == i in the next cycle.
- Maximum wait is one critical section execution of the other thread.

6. Code Walkthrough

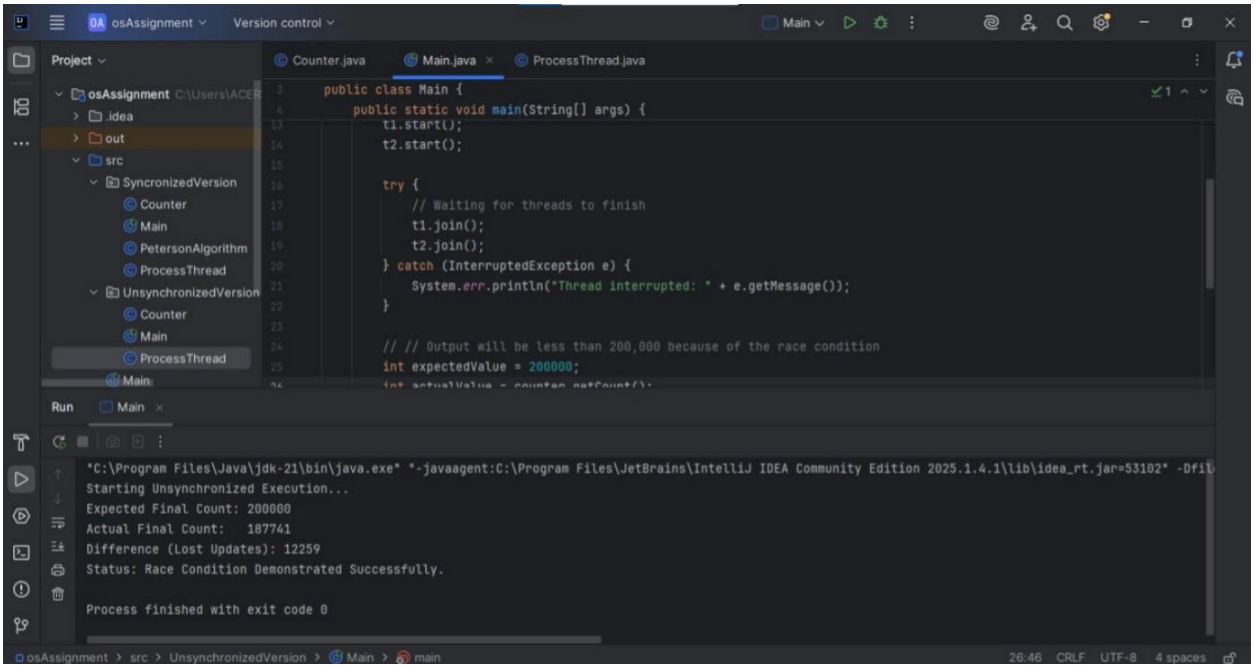
Section	Explanation
Flag 2	Each treads declared intent to enter the critical section.
Turn	Ensures fairness by alternating turns.
enter(int i)	Sets flag, yields turn, and waits.
While (flag[other]	Busy-wait until safe to enter.
exit(int i)	Resets flag, allowing other thread to proceed.
Increment() method	Wraps critical section (count++) with lock/unlock calls.

7.Screenshots of program output and performance comparison.

1.synchronized version



2. unsynchronized version



8. Conclusion and Reflections

Peterson's Algorithm is an elegant software-only solution that demonstrates how synchronization can be achieved without hardware or OS support. It satisfies all three critical-section requirements but is limited to two threads. It also uses busy waiting, which may waste CPU cycles.

9.References:

- Peterson, G. L. (1981). "Myths About the Mutual Exclusion Problem." Information Processing Letters.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts. Wiley.